

devreport: Turning a Project Folder into a LaTeX Report or Slide Deck

Celibistrial

October 2026

Abstract

Student builders at hackathons, in coursework and on side projects have to write up what they built, and most would rather keep building. `devreport` takes a messy project folder (dev logs, notes, CSV tables, screenshots, a `.zip` or a GitHub link) and returns a compiled PDF report or Beamer slide deck of a chosen length, generated live in the browser. The design splits the work in two. Deterministic Node.js code (`collect.js`) parses git history, logs, tables, notes and images into CSV files and a `facts.json` summary. A headless Claude Code agent then reads those facts, writes the LaTeX, compiles it with `tectonic`, fixes errors and recompiles. Charts are `pgfplots` figures that read the computed CSVs, so the agent never types a number into a chart. The agent runs with a locked-down tool allowlist that only lets it write inside the job folder, and it asks the user questions only when the files do not say what the project is or who it is for. End-to-end runs through the real interface produced a report from a GitHub link in about two minutes and a slide deck from a zip in about one minute.

1 Introduction

At the end of a hackathon or a course project, the code usually exists but the write-up does not. The material for it is scattered: a git history, terminal logs, a notes file, a few screenshots and perhaps a CSV of results. Turning that into a structured report or a presentation is slow work that comes at the point where the builder has the least time left. `devreport` is aimed narrowly at this case. Its users are student builders who need to describe a hackathon, coursework or side project, and it is not meant to turn an arbitrary idea into a document.

The narrow scope also answers a common objection to tools of this kind, that they are only a wrapper around a language model. In `devreport` the analysis of the project is done by ordinary, tested code, and the language model is used for what it is good at: reading the extracted facts and writing prose and LaTeX around them. A second goal is that the output should not contain made-up data. Every chart plots a CSV that the collector computed, and stated numbers come from `facts.json`. Commit counts and lines of code are not treated as evidence of quality; they appear only as a timeline or an appendix sentence.

2 System Overview

The user opens a single-page interface, drops files or a zip, or pastes a public GitHub URL, chooses Report or Slides, picks a length (5–25 slides or 2–10 pages) and, for slides, a theme or an uploaded PowerPoint template. Each request becomes a job folder `jobs/<id>/` on the server, and everything that follows happens inside that folder. Table 1 lists the components.

The pipeline has five stages. First, the server saves the upload into `input/`, unpacking a zip or cloning a GitHub repository. Second, `collect.js` turns the inputs into CSV files and `facts.json`. Third, the server spawns Claude Code headless (`claude -p`) with the job folder as

Table 1: Components of devreport and what each one does.

File	Job
<code>collect.js</code>	Deterministic parsing of the inputs into <code>data/*.csv</code> and <code>facts.json</code>
<code>pptx.js</code>	Extracts colours, fonts, layout boxes, media and the thumbnail from a <code>.pptx</code> template into <code>template/</code>
<code>server.js</code>	Uploads, zip and GitHub intake, runs the collector and <code>claude -p</code> , streams progress over server-sent events
<code>index.html</code>	The whole user interface in one file: drop zone, options, live feed and PDF viewer
<code>test.js</code>	<code>node:test</code> tests for the collector, the template extractor and the server’s validators
<code>.claude/skills/devreport/</code>	The pipeline rules the agent follows, tested chart snippets, the prompt and the themes

its working directory, so the project’s devreport skill is loaded from the repository. The agent reads everything, and if something essential is missing it writes `questions.json` and stops. Fourth, if questions were asked, the user answers or skips them in the feed, the answers are saved as `input/answers.md`, and the same agent session resumes with `--resume`. Fifth, the agent writes `main.tex`, compiles it with `tectonic`, fixes any errors, runs a humanizer pass on the prose and recompiles. The server then shows `main.pdf` inline, with the `.tex` available to download.

The server is written with the Node.js standard library only and needs no `npm install`. It forwards the agent’s stream-json output to the browser as server-sent events, and the interface folds these steps into one line per phase (uploading, analysing, reading, writing, compiling, polishing, done), so a run of one to three minutes reads as progress rather than a frozen page.

3 Implementation

3.1 The collector

`collect.js` handles each input type with fixed rules. From git history (`git log --numstat`) it computes commits per day, milestones, lines added and removed, the most-edited files, commit hours and commit types, with vendored paths, lockfiles and build directories excluded. Milestones are at most eight commits: the first commit, tagged commits, then the largest feature commits spread over equal time windows of the project. Log files are scanned with regular expressions for ISO, syslog and `[HH:MM:SS]` timestamps and for ERROR, WARN and INFO levels; repeated errors are grouped by replacing numbers, hex strings and IDs with placeholders, so that two database timeouts with different durations count as one pattern. For user CSV and JSON tables, the collector detects each column as a date, number or category and picks a chart from that: a date and a number give a line chart, a category and a number give a bar chart, and an all-numeric table whose first column strictly increases (such as an epoch) gives one line chart per metric group. Notes are passed through as text and images are copied in with stable IDs.

3.2 Charts that cannot show invented data

Every chart in a document is a pgfplots figure of the form `\addplot table{data/x.csv}`. The agent chooses which charts to include and writes the axis code from tested snippets, but the

numbers come from the CSV on disk. The CSV writer strips commas, quotes and newlines from cells because pgfplots does not understand CSV quoting.

3.3 The locked-down agent

The backend is the real Claude Code CLI running headless, not the Claude API, and each user runs it with their own Claude Code login. It is started with this tool allowlist:

```
Read,Write(./**),Edit(./**),Bash(tectonic:*),Skill
```

so it has no general shell and can only run `tectonic`. The write rules are relative to the working directory. When tested with `claude -p`, a write to `../escape.txt` or `/tmp/x` was denied, while `main.tex` and `template/x.sty` were allowed; a bare `Write` permission allowed the escape. Uploaded content is treated as data and never as instructions.

Intake has its own checks. Uploads are capped at about 50 MB, and a zip is rejected if the sizes listed by `unzip -l` add up to more than about 200 MB, which guards against zip bombs. Entries that resolve outside `input/` are rejected and symlinks are deleted afterwards, so the agent's `Read` cannot follow one out of the job. GitHub URLs must match a strict pattern and are cloned through `execFile` without a shell, with `GIT_TERMINAL_PROMPT=0` and a timeout of about 60 seconds.

3.4 Themes and templates

Reports use one standard article layout. Slides can use Paper, Midnight, Metropolis, Moloch, Focus, Trigon or Madrid. Each theme is a small adapter `.sty` that exposes the same colour names, chart styles and type sizes, and `main.tex` only says `\input{theme.tex}`. Switching the theme of a finished deck therefore rewrites one line and reruns `tectonic` without the agent, which takes about 1.2 s, and each compiled theme is cached. For a custom PowerPoint template, `pptx.js` unzips the `.pptx` and reads the colour palette and fonts from the theme XML, the title and body box positions from the slide layouts, and the media and thumbnail. The agent writes a matching Beamer theme from this, compiles a test slide, compares it with the thumbnail and adjusts.

3.5 Timeline

The whole project was built on 2 October 2026 for the FirstCommit hackathon. Table 2 lists the milestones the collector picked from the git history.

4 Results

The full pipeline was tested end to end through the real interface: a GitHub link produced a report in about two minutes, and a zip produced a slide deck in about one minute at a cost of about \$0.5. The question flow, the length option and the PowerPoint import were also tested through the interface, and sample PDFs, including one devreport generated about itself, are committed under `samples/`.

The deterministic parts are covered by `node:test` tests in `test.js` that build small fixture repositories and folders. They check that git history, logs, tables, notes and images are collected into the expected CSV rows and facts; that vendored-only commits, nested repositories and authors with a single commit stay out of the statistics; that milestones are spread across the project span and include tagged commits; that syslog timestamps get the right year across a December-to-January rollover; that an epoch-indexed results table gets one line chart per metric group; that empty input never throws; and that `pptx.js` extracts palette, fonts and layout boxes while rejecting non-`.pptx` files and entries that escape the output folder. Further

Table 2: Milestones from the git history, in order.

Date	Event
2026-10-02	Added the project plan and handoff notes
2026-10-02	Added the server: multipart upload, zip and GitHub intake, collector and agent run, progress over server-sent events
2026-10-02	Added the UI: drop zone, repo link, report or slides, themes, questions, live feed and PDF view
2026-10-02	Collector converts 16-bit PNGs, strips commas from CSV labels and adds day counts
2026-10-02	Accepted a <code>.pptx</code> template for slides and added a template theme card
2026-10-02	Added <code>scripts/fill.py</code> to measure how full each slide body is
2026-10-02	Revision can add files, a zip, a repo or notes as new material after the first draft
2026-10-02	UI shows one stage at a time (form, feed, full-size viewer with sidebar), a mobile pass, and “Add material” in revise

tests cover the server’s validation of revision feedback, lengths, questions, answers and theme names.

5 Discussion

The main design decision was to keep the language model away from the analysis. Putting the parsing in `collect.js` made the numbers testable and kept the agent’s role to writing, which also gives a clear answer to how the system works. A second decision was to ask questions only when needed. An early plan had a separate gap-check call and a fixed question form; this was replaced by letting the one agent run write `questions.json` when it cannot tell what the project does or who it is for, with at most three questions asked once per job. Most repositories with a README get none.

The system has known limits. It runs locally, one job at a time, and there is deliberately no public hosted version, since that would run strangers’ jobs on one person’s Claude Code subscription. Template import copies colours, fonts, the logo and rough layout, but not shapes, gradients or animations, and missing fonts fall back to a close TeX Gyre font. Re-encoding 16-bit PNGs uses `sips`, which is macOS only. Uploaded text still reaches an agent that can write files in the job folder; the allowlist contains this, but the tool should only be run on trusted material. Two small issues remain open: error grouping turns `sqlite3` into `sqlite<n>`, and zips with a top-level folder unpack one level deeper than needed.

A Repository Statistics

The repository has 58 commits by Celibistrial, all made on one active day (2026-10-02), with 3794 lines added and 492 removed.